Etec
Dr. Adail Nunes da Silva
Taquaritinga

55 anos CPS Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO
SÃO PAULO SÃO TODOS

Programação e Algoritmos

Explorando o ambiente do Visual Studio Code para o C#

Wagner Cesar Vieira
wagner.vieira01@cps.sp.gov.br

CPS Centro Paula Souza
SÃO PAULO GOVERNO DO ESTADO

Roteiro de aula

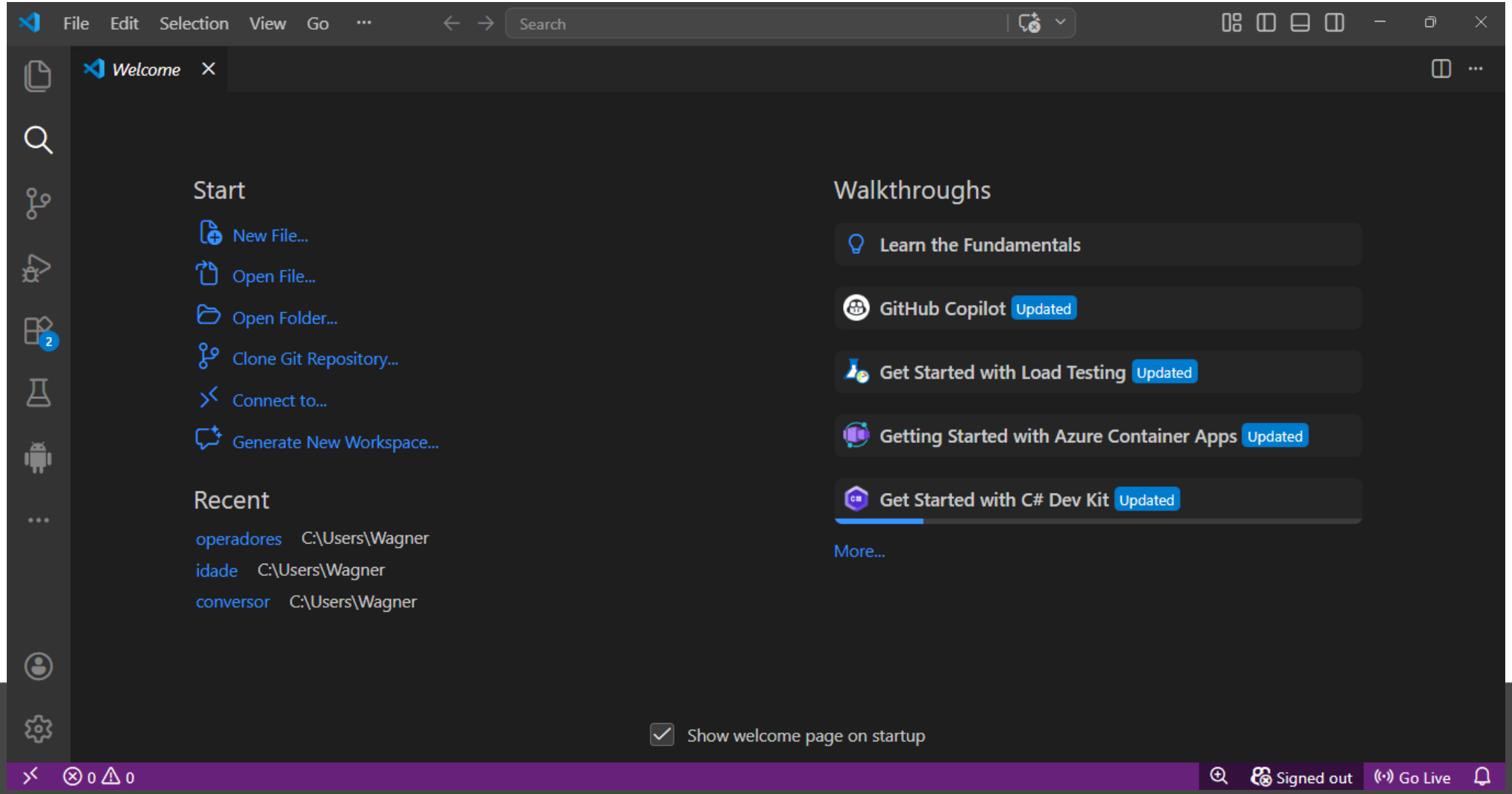
- ❑ Apresentando o Visual Studio Code
- ❑ Uso do terminal (Shell) para iniciar projetos
- ❑ Tipos de dados e formatação
- ❑ Operadores matemáticos, relacionais e lógicos
- ❑ Conversão de tipos
- ❑ Estruturas condicionais (if-else e switch-case)
- ❑ Estruturas de controle (while, do-while, for, foreach)
- ❑ Exercício de aplicação
- ❑ Lista de exercícios

Etec
Dr. Adail Nunes da Silva
Taquaritinga

Visual Studio Code

55 anos CPS Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO SÃO PAULO SÃO TODOS



Iniciando o primeiro projeto Aplicação Console

Prompt de Comando

```
Microsoft Windows [versão 10.0.26200.8039]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Wagner>dotnet new console -o operador
O modelo "Aplicativo do Console" foi criado com êxito.

Processando ações pós-criação...
Restaurando C:\Users\Wagner\operador\operador.csproj:
A restauração foi bem-sucedida.



C:\Users\Wagner>code -r operador
```

- **dotnet** -> compilador do C#
- **new console** -> cria uma aplicação console (terminal)
- **code –r nome** -> chama o Visual Studio Code com a pasta do projeto aberta.

Exemplo de código fonte em C#

Etec Dr. Adail Nunes da Silva Taquaritinga

55 anos CPS Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO SÃO PAULO SÃO TODOS

File Edit Selection View ··· media

EXPLORER

MEDIA
- bin
- obj
- media.csproj
- media.sln
- Program.cs

OUTLINE
TIMELINE
SOLUTION EXPLORER

Program.cs

```
// Algoritmo que calcula a média
double num1, num2, media = 0;

Console.WriteLine("Algoritmo que calcula a média de dois valores");

Console.Write("Digite o 1º valor: ");
//num1 = Double.Parse(Console.ReadLine());
num1 = Convert.ToDouble(Console.ReadLine());

Console.Write("Digite o 2º valor: ");
// num2 = Double.Parse(Console.ReadLine());
num2 = Convert.ToDouble(Console.ReadLine());

// Calcula a média
media = (num1 + num2) / 2;

//Exibe o resultado no terminal
Console.WriteLine($"A média dos valores é {media}");
```

Microsoft .NET

Após a codificação, salvamos o projeto e utilizamos a tecla de atalho **F5** para compilar e executar o código.

Saída no console

```
C:\Users\Wagner\media>dotnet run
Algoritmo que calcula a média de dois valores
Digite o 1° valor: 8,25
Digite o 2° valor: 9,35
A média dos valores é 8,8

C:\Users\Wagner\media>
```

Identificadores

- São nomes utilizados para identificar os elementos nos seus programas. Esses elementos podem ser:
- **namespace** (espaço de nomes), **classes**, **métodos**, **variáveis**, etc.

Podemos usar apenas letras, número e sublinhado para compor o nome dos identificadores.

- **Constantes**

const <tipo> <nomeDaConstante> = <valor>;

- Exemplos:
- const int numLinhas = 3;
- const int numColunas = 2;
- const double Pi = 3.14159265358979323846;

Variáveis em C#

- Uma variável é definida como um lugar onde podemos armazenar dados temporariamente e usá-los posteriormente.
- Uma variável poderá armazenar um número inteiro, de ponto flutuante, um caractere, uma cadeia de caracteres (String) , um valor booleano, etc.
- O nome de uma variável pode conter letras, número e '_' (não pode haver sinais de pontuação e espaços em branco).
- Define o tipo de dados da variável

✓ integer

✓ byte, short, long

✓ float, double, decimal

✓ char, string

✓ bool: true (verdadeiro), false (falso)

Declaração de variáveis

- int a, idade;
- char ch, caracter;
- float f, valor;
- double d;
- string s;

Toda variável deverá ser declarada explicitamente antes de ser usada

Variáveis em C#

Tipo de dados	Intervalo
byte	0 ..255
sbyte	-128 ..127
short	-32,768 ..32,767
ushort	0 ..65,535
int	-2,147,483,648 ..2,147,483,647
uint	0 ..4,294,967,295
long	-9,223,372,036,854,775,808..9,223,372,036,854,775,807
ulong	0 ..18,446,744,073,709,551,615
float	-3.402823e38 ..3.402823e38
double	-1.79769313486232e308 ..1.79769313486232e308
decimal	-79228162514264337593543950335..79228162514264337593543950335
char	U+0000 .. U+ffff

Palavras reservadas de C#

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explict	extern	false	finally

fixed	float	for	foreach	get
goto	if	implict	in	int
interface	internal	is	lock	long
namespace	new	null	object	operator
out	override	params	private	protected

Palavras reservadas de C#

public	readonly	ref	return	sbyte
sealed	set	short	sizeof	stackalloc
static	string	struct	switch	this
throw	true	try	typeof	uint
ulong	unchecked	unsafe	ushort	using

Atribuição de valores

- Podemos atribuir o valor a uma variável usando o símbolo = (operador de atribuição). Exemplos:

int a = 1; // atribui o valor 1 a variável inteira a

float f = -8.76F; // observe o sufixo F

char c = "a"; // atribui o caracter a na variável c

double num = 2.55; // atribui o valor 2.55 na variável num

Propriedades de objetos

- value.text = 1; → erro
- value.text = "1"; → **ok**, porém o número um está sendo tratado como texto e não como número;
- value.text = **string.Parse**(1); · o número um é convertido de inteiro para uma string pelo uso do método parse da classe string.

Tipos, variáveis e valores

As informações armazenadas em um tipo podem incluir o seguinte:

- O espaço de armazenamento que uma variável do tipo requer.
- Os valores mínimo e máximo que ele pode representar.
- Os membros (métodos, campos, eventos e etc.) que ele contém.
- O tipo base do qual ele herda.
- O local no qual a memória para as variáveis será alocada em tempo de execução.
- Os tipos de operações que são permitidos.

C# Copiar

```
int a = 5;
int b = a + 2; //OK

bool test = true;

// Error. Operator '+' cannot be applied to operands of type 'int' and 'bool'.
int c = a + test;
```

Exemplo de código fonte em C#

Etec Dr. Adail Nunes da Silva Taquaritinga

55 anos CPS Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO SÃO PAULO SÃO TODOS

File Edit Selection View ··· temperatura

EXPLORER

TEMPERATURA
- bin
- obj
- Program.cs
- temperatura.csproj
- temperatura.sln

OUTLINE
TIMELINE
SOLUTION EXPLORER

Program.cs

```
// Converte temperaturas de °C para °F e °K
// Celsius (°C), Fahnhereit (°F) e Kelvin (°F)
double celsius, fahnhereit, kelvin = 0;

Console.WriteLine("Algoritmo que converte temperaturas");
Console.Write("Informe a temperatura em °C: ");
celsius = Convert.ToDouble(Console.ReadLine());

fahnhereit = celsius * 1.8 + 32;
kelvin = celsius + 273.15;

Console.WriteLine(@$"A temperatura em Celsius é {celsius}°C
A temperatura em Fahnhereit é {fahnhereit}°F
A temperatura em Kelvin é {kelvin}°K");
```

```
C:\Users\Wagner\temperatura>dotnet run
Algoritmo que converte temperaturas
Informe a temperatura em °C: 100
A temperatura em Celsius é 100°C
A temperatura em Fahnhereit é 212°F
A temperatura em Kelvin é 373,15°K

C:\Users\Wagner\temperatura>
```

Tipos, variáveis e valores

Quando declara uma variável ou constante em um programa, você deve especificar seu tipo ou usar a palavra-chave **var** para permitir que o compilador **infira** o tipo. O exemplo a seguir mostra algumas declarações de variáveis que usam tipos numéricos internos e tipos complexos definidos pelo usuário:

Etec
Dr. Adail Nunes da Silva
Taquaritinga

Tipo	Tamanho (em bytes)	Tipo .NET	Descrição
byte	1	Byte	Sem sinal (0-255)
char	2	Char	Caractere Unicode
bool	1	Boolean	Booleano (true=Verdadeiro ou false=Falso)
short	2	Int16	Com sinal (-32.768 a 32.767)
int	4	Int32	Com sinal (-2.147.483.648 e 2.147.483.647)
float	4	Single	Números de ponto flutuante. Armazena os valores de aproximadamente +/- 1,5*10-45 até aproximadamente +/-3,4*10^38 com sete número significativos. Usa o sufixo F
double	8	Double	Números de ponto flutuante de precisão dupla (com 15 a 16 números significativos)
decimal	16	Decimal	Valores monetários. Precisão fixa até 28 dígitos e a posição do ponto decimal. É normalmente usado em cálculos financeiros. Exibe sufixo “m” ou “M”
long	8	Int64	Números inteiros com sinal de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807

Exemplos de tipos de dados em C#

Etec Dr. Adail Nunes da Silva Taquaritinga

55 anos CPS Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO SÃO PAULO SÃO TODOS

File Edit Selection View ··· operadores

EXPLORER

OPERA...

- bin
- obj
- operadores.csproj
- operadores.sln
- Program.cs

OUTLINE

TIMELINE

SOLUTION EXPLORER

Program.cs

```
// Declaração de variáveis
// Números inteiros
byte a = 0;
byte b = 255;
short c = 12345;
int d = 98578;

// Números flutuantes
float num1 = 12555.42F;
double num2 = 12555.425263F;
long num3 = -89597854522255;
decimal salario = 10245.20M;

// Caracteres e strings
char letra = 'W';
string texto = "Um texto grande!";

// Valor lógico
bool chuva = true;

Console.WriteLine(@$"Os valores das variáveis são:
O valor de a é {a}
O valor de b é {b}
O valor de c é {c}
O valor de d é {d}
O valor de num1 é {num1}
O valor de num2 é {num2}
O valor de num3 é {num3}
O valor de salario é {salario}
O valor de letra é {letra}
O valor de texto é {texto}
O valor de chuva é {chuva}");
```

```
C:\Users\Wagner\operadores>dotnet run
Os valores das variáveis são:
O valor de a é 0
O valor de b é 255
O valor de c é 12345
O valor de d é 98578
O valor de num1 é 12555,42
O valor de num2 é 12555,4248046875
O valor de num3 é -89597854522255
O valor de salario é 10245,20
O valor de letra é W
O valor de texto é Um texto grande!
O valor de chuva é True

C:\Users\Wagner\operadores>
```

Tabela de formatação tipos numéricos

Especificador de Formato	Descrição	Exemplos	Saída
"C" ou "c"	Moeda	Console.Write("{0:C}", 2.5);	$2.50
		Console.Write("{0:C}", -2.5);	($2,50)
D ou d	Decimal	Console.Write("{0:D5}", 25);	00025
"E" ou "e"	Científico	Console.Write("{0:E}", 250000);	2,500000E+005
F ou f	Ponto fixo	Console.Write("{0:F2}", 25);	25,00
		Console.Write("{0:F0}", 25);	25
"G" ou "g"	Geral	Console.Write("{0:G}", 2.5);	2.5
"N" ou "n"	Número	Console.Write("{0:N}", 2500000);	2.500.000,00
"X" ou "x"	Hexadecimal	Console.Write("{0:X}", 250);	FA
		Console.Write("{0:X}", 0xffff);	FFFF

Operadores matemáticos

++ , --	Incremento, decremento
+ , - , * , /, %	adição, subtração, multiplicação, divisão, módulo (resto)

Exemplo:

```
int a = 7;
int b = 3;
int x,y;
x = a / b;
y = a % b;
```

```
int x, y;
x = 23;
y = x++;            // operador pós-fixado
Console.WriteLine(x + " " + y); // 23 24

x = 23;
y = ++x;            // operador pré-fixado
Console.WriteLine(x + " " + y); // 24 e 24
```

Operadores relacionais e lógicos

Um operador booleano é um operador que faz um cálculo cujo resultado é true (verdadeiro) ou false (falso)

Operador	Ação
==, !=	Igual a, diferente de
> , >= , < , <=	Maior que, maior ou igual a, menor que, menor ou igual a
&&	“E” lógico (AND)
||	“OU” lógico (OR)
!	“NÃO” lógico

Exemplo:
bool porcentagemValida = (porcentagem>=0) **&&** (porcentagem<=100);

Conversão de tipos

É possível converter um tipo de dados para outro tipo no C#. Podemos realizar conversão implícita de tipos (ou seja, o C# realiza a conversão de forma segura automaticamente) ou explícita (o programador utiliza funções pré-definidas - "cast"), ou ainda usar classes auxiliares.

Vamos ver um exemplo de conversão implícita:

```
int i = 10;
Console.WriteLine("O número é " + i);
```

O conteúdo da variável i, do tipo inteira, é convertida implicitamente para o tipo string ao passá-la como **parâmetro** para o método **WriteLine** da classe Console.

Tabela de conversão de tipos


```
int i = 90;
float f = 22.100f;
double d = 1450.33;
bool a = true;
```

```
Console.WriteLine(i.ToDouble());
Console.WriteLine(f.ToDecimal());
Console.WriteLine(d.ToInt16());
Console.WriteLine(a.ToString());
```

Método	Conversão Realizada
ToBoolean	Converte um tipo em valor booleano, se possível
ToByte	Converte um tipo em Byte.
ToChar	Converte um tipo em caracter Unicode, se possível.
ToDecimal	Converte um ponto-flutuante ou um inteiro em decimal.
ToDouble	Converte um tipo em Double
ToInt16	Converte um tipo em inteiro de 16 bits.
ToInt32	Converte um tipo em inteiro de 32 bits.
ToInt64	Converte um tipo em inteiro de 64 bits.
ToString	Converte um tipo em String.

Conversão de Tipos com a classe Convert

A classe System.Convert permite converter um tipo base em outro tipo de dados base usando seus métodos.

Exemplo:

```
int x = 15
double y;
y = Convert.ToDouble(x);
Console.WriteLine("Valor convertido: {0} do tipo {1}", y, y.GetType());
Console.WriteLine("Valor não convertido: {0} do tipo {1}", x, x.GetType());
```

Conversão de Tipos com o método Parse


```
String s = Console.ReadLine();
/* quando temos um número representado por uma string, não podemos fazer
   operações aritméticas com ele abaixo estão definidas as variáveis i, f e d */
int i;
i = int.Parse(s); //converte uma variável do tipo String em um inteiro

float f;
f = float.Parse(s); //converte uma variável do tipo String em um float

double d;
d = double.Parse(s); // converte uma variável do tipo String em um double
```

Estrutura condicional if – else – else if (se – senão)

O if avalia uma expressão lógica booleana e qualquer outro tipo será acusado como erro pelo compilador. Se o resultado for verdadeiro, o bloco de código dentro do if será executado; caso contrário, o controle é passado para a próxima declaração após o if, a declaração if tem três formas básicas:

1. IF Simples

```
if (expressão)
    {
        Declaração
    }
```

2. IF - ELSE

```
if (expressão)
{
    Declaração
}
else
{
    Declaração
}
```

3. IF Aninhado

```
if (expressão)
{
    Declaração
}
else if (expressão)
{
    Declaração
}
```

Exemplos


```
// Exemplo1
int a = 0;
int b = 1;
if ( a < b )
{
    Console.WriteLine("B é maior");
}
else
{
    Console.WriteLine("A é maior");
}
```

```
//Exemplo2
int a = 0;
int b = 1;
if ( a < b )
{
    Console.WriteLine("B é maior");
}
else if ( a > b )
{
    Console.WriteLine("A é maior");
}
else
/* e finalmente a condição de igualdade deveria
ser satisfeita */
{
    Console.WriteLine("A é igual a B");
}
```

Estrutura condicional Switch (escolha)

A declaração switch avalia uma expressão cujo resultado pode ser dos tipos sbyte, byte, short, ushort, int, uint, long, ulong, char, string ou enum, e este por sua vez é comparado com cada uma das seções case que constituem o switch. Vejamos a sua sintaxe:

```
switch(expressão)
{
    case constante1:
        declaração 1;
        break;
    case constante2:
        declaração 2;
        break;
    ...
    default:
        declarações;
        break;
}
```

```
char letra = 'b';
string resposta = " ";
switch( letra ) {
    case 'a':
        resposta = "Resposta errada!";
        break;
    case 'b':
        resposta = "Resposta correta!";
        break;
    default:
        resposta = "Resposta inválida!";
    }
    meuLabel.Text = "resposta";
```

Exemplo de Switch (Case)


```
int opcao;
Console.WriteLine("Digite a opção desejada: ");
opcao = int.Parse(Console.ReadLine());
switch (opcao)
    {
    case 0:
        Console.WriteLine("A é igual a B");
        break;
    case 1:
        Console.WriteLine("A é menor do que B");
        break;
    case 2:
        Console.WriteLine("A é maior do que B");
        break;
    }
    Console.ReadKey();
```

1. Em C#, é obrigatório que cada seção case tenha uma declaração break.

2. A seção default, que é avaliada caso nenhuma das seções case for verdadeira,
não é obrigatória.

3. Não pode existir mais de uma seção case com a mesma constante.

Estrutura de repetição
Laço while (enquanto)

O laço **while** é usado quando não sabemos o número de vezes que devemos executar um bloco de código, mas apenas a condição que deve ser satisfeita para executar o bloco dentro do **while**. Essa condição é uma expressão booleana que deverá ser verdadeira para garantir pelo menos a primeira ou a próxima iteração.

Exemplo:

```
int i = 95;          // declara a variável i e atribui o valor 95
while ( i< 100) // testa a condição usando o laço While (enquanto)
    {
        Console.WriteLine("  {0}", i); // Escreve no console o valor de i
        i++;    // incrementa a variável i, aumentando de 1 em 1
    }
```

C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

```
95
96
97
98
99
```

Exemplos de While


```
int contador = 0;
int valor = 0;
while ( contador < 10 ) {
    valor += 10;
    contador ++;
}
meuLabel.Text = valor.ToString( );
int contador = 0;
int valor = 0;
while ( contador < 10 ) {
    valor += 10;
    contador ++;
}
meuLabel.Text = valor.ToString( );
```

```
int n = 0;
while (n < 5)
{
    Console.WriteLine(n);
    n++;
}
```

C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

```
0
1
2
3
4
```

O laço do ... while (faça – enquanto)

Este tipo de laço é usado quando queremos que um bloco de código seja executado pelo menos uma vez. A condição a ser satisfeita se encontra no fim do bloco de código e não no começo, como no caso dos laços for e while.

Exemplo:

```
int i = 95;
do
{
    Console.WriteLine(" {0}", i);
    i++;
} while ( i< 100)
```

C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

```
95
96
97
98
99
```

No exemplo acima, se o valor da variável i fosse igual a 100 antes da execução de do/while, o laço seria executado pelo menos uma vez, o que não aconteceria se usássemos while.

Estrutura de repetição
Laço for (para)

O laço for segue o mesmo estilo das linguagens C/C++, e a sua sintaxe tem as seguintes características:

- Uma variável de inicialização pode ser declarada dentro do for.
- Uma condição avalia uma expressão para abandonar o for ou executá-lo
- de novo.
- Uma expressão incrementa o valor da variável de inicialização.

Exemplo: for(int i=0 ; i<=5; i++)

```
// (inicialização ;   teste de condição ;   incremento)
{
    Console.WriteLine ("Iteração número {0}", i);
    if (i == 3)
        break;
}
```

Para abandonar o laço antes que a condição for seja falsa, usa-se a palavra reservada break.

Estrutura de repetição
Laço for (para)

A palavra reservada continue permite que o fluxo de execução da iteração corrente seja abandonado, mas não o laço, e a iteração seguinte dê início no topo do laço, uma vez que a condição do for seja satisfeita.

Exemplo:

```
for(int i=0; i<=5; i++) {
    Console.WriteLine("Iteração número {0}", i);
    if (i == 3)
    break;
}
for(int i=0; i<=5; i++) {
    Console.WriteLine("Iteração número {0}", i);
    if (i == 3)
    continue;
    // a declaração a seguir não será executada quando i==3
    Console.WriteLine ("Iteração número {0}", i +2 );
}
```

```
Iteração número 0
Iteração número 1
Iteração número 2
Iteração número 3
```

```
Laço for com continue!
Iteração número 0
Iteração número 2
Iteração número 1
Iteração número 3
Iteração número 2
Iteração número 4
Iteração número 3
Iteração número 4
Iteração número 6
Iteração número 5
Iteração número 7
```

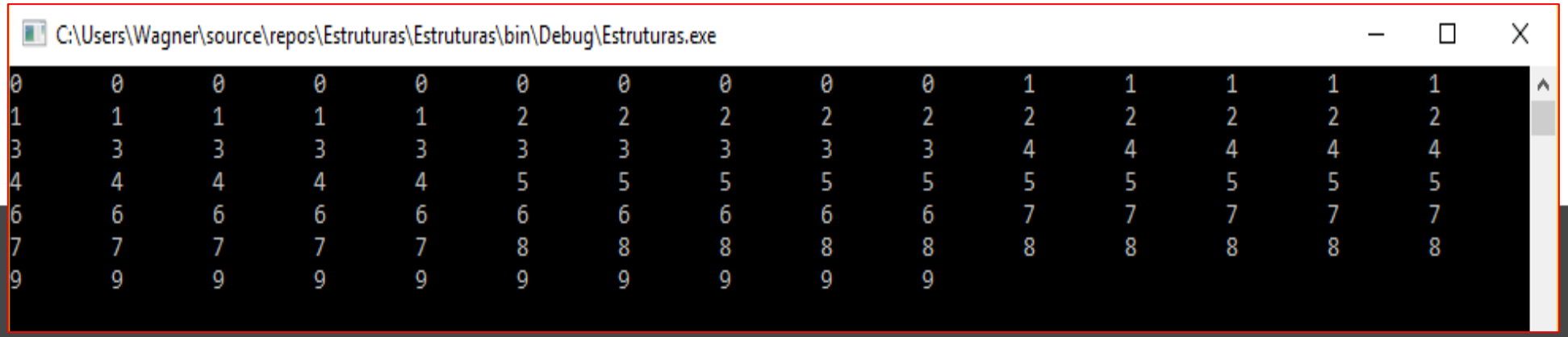
Laços aninhados

Laços aninhados são laços dentro de laços que podem ser usados, por exemplo, (como já o fizemos anteriormente) para varrer **arrays** multidimensionais.

Exemplo:

```
int i, j;
for (int i=0; i < 10; i++)
{
    for (int j=0; j < 10; j++)
    {
        Console.Write("{0}\t", i, j) ;
    }
}
```

Se uma declaração break estiver no laço interno, este será abandonado e o controle será passado para o laço externo; mas se estiver no laço externo, os dois laços serão abandonados e o controle passará para a próxima declaração após o laço.



Laços foreach / in

Este tipo de laço é usado para varrer arrays ou coleções.

```
class Class1
{
    static void Main(string[] args)
      {
          string[ ] nomes = { "Ana", "Maria", "João", "José" };
          foreach (string pessoa in nomes)
          {
              Console.WriteLine(" {0} ", pessoa);
          }
          Console.ReadKey();
      }
}
```

C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

```
Ana
Maria
João
José
```

Lista de tarefas

1. Criar repositório no GitHub;
2. Iniciar a resolução da lista de exercícios;
3. No final das aulas, subir para o repositório.

Etec
Dr. Adail Nunes da Silva
Taquaritinga

1. Criar repositório no GitHub

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

1 **General**

Owner *

wcrvieira /

Repository name *

algoritmos_cs

algoritmos_cs is available.

Great repository names are short and memorable. How about refactored-broccoli?

Description

Algoritmos com .Net e C Sharp no modo console

45 / 350 characters

2 **Configuration**

Choose visibility *
Choose who can see and commit to this repository — Public

Add README
READMEs can be used as longer descriptions. About READMEs — On

Add .gitignore
.gitignore tells git which files not to track. About ignoring files — Dotnet

Add license
Licenses explain how others can use your code. About licenses — No license

Create repository

Comandos GIT

// Configura o usuário do GitHub – variável local

git config –global user.name “seu_usuário”

// Configura o e-mail do GitHub – variável local

git config –global user.email “seu_email_do_github”

// Configura o repositório do GitHub – criado anteriormente

git remote add origin “seu_repositório_git”

Comandos GIT

// Adiciona os arquivos do projetos no git local

git add .

// Confirma a inclusão dos arquivos para enviar

git commit –m “Sua mensagem inicial”

// Envia os arquivos do local para o remoto (GitHub)

git push –u origin **main ou master** --force

Comandos GIT

// Limpa o repositório remoto

```
git remote remove origin
```

// Verifica o status dos arquivos

```
git status
```

// Lista as configurações locais do Git

```
git config --list
```

Criando projeto no console

// Criando e acessando a pasta do projeto

Prompt de Comando

```
Microsoft Windows [versão 10.0.26200.8246]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\Wagner>mkdir projeto

C:\Users\Wagner>cd projeto

C:\Users\Wagner\projeto>
```

// Criando o projeto .Net via console

dotnet new console -o area

Criando projeto no console

// Abrindo o projeto no Visual Studio Code

```
C:\Users\Wagner\projeto>code -r area_circulo
```

Exercícios – Estruturas sequenciais

1. Faça um programa para ler o valor do raio de um círculo, e depois mostrar o valor da área deste círculo com quatro casas decimais conforme exemplos.

- ❑ Fórmula da área: area = π . raio2
- ❑ Considere o valor de π = 3.14159

2. Fazer um programa para ler quatro valores inteiros A, B, C e D. A seguir, calcule e mostre a diferença do produto de A e B pelo produto de C e D segundo a fórmula: DIFERENCA = (A * B - C * D).

3. Fazer um programa que leia três valores com ponto flutuante de dupla precisão: A, B e C. Em seguida, calcule e mostre:

a) a área do triângulo retângulo que tem A por base e C por altura.
b) a área do círculo de raio C. (pi = 3.14159)
c) a área do trapézio que tem A e B por bases e C por altura.
d) a área do quadrado que tem lado B.
e) a área do retângulo que tem lados A e B.

Exercícios – Estruturas condicionais

4. Fazer um programa para ler um número inteiro, e depois dizer se este número é negativo ou não e se é divisível por ele mesmo.

5. Fazer um programa para ler um valor em reais com duas casas decimais e calcular o valor que esta pessoa deve pagar de impostos, conforme a tabela abaixo:

Renda	Imposto de Renda
de 0.00 a R$ 2000.00	Isento
de R$ 2000.01 até R$ 3000.00	8 %
de R$ 3000.01 até R$ 4500.00	18 %
acima de R$ 4500.00	28 %

6. Você deve fazer um programa que leia um valor qualquer e apresente uma mensagem dizendo em qual dos seguintes intervalos ([0,25], (25,50], (50,75], (75,100]) este valor se encontra. Obviamente se o valor não estiver em nenhum destes intervalos, deverá ser impressa a mensagem "Fora de intervalo".

Exercícios – Estruturas de repetição - While

7. Escreva um programa que repita a leitura de uma senha até que ela seja válida. Para cada leitura de senha incorreta informada, escrever a mensagem "Senha Invalida". Quando a senha for informada corretamente deve ser impressa a mensagem "Acesso Permitido" e o algoritmo encerrado. Considere que a senha correta é o valor **@2026**.

8. Um Posto de combustíveis deseja determinar qual de seus produtos tem a preferência de seus clientes. Escreva um algoritmo para ler o tipo de combustível abastecido (codificado da seguinte forma: 1.Álcool 2.Gasolina 3.Diesel 4.Fim). Caso o usuário informe um código inválido (fora da faixa de 1 a 4) deve ser solicitado um novo código (até que seja válido). O programa será encerrado quando o código informado for o número 4. Deve ser escrito a mensagem: "MUITO OBRIGADO" e a quantidade de clientes que abasteceram cada tipo de combustível.

Exercícios – Estruturas de repetição - For

9. Leia um valor inteiro X (1 <= X <= 1000). Em seguida mostre os ímpares de 1 até X, um valor por linha, inclusive o X, se for o caso.

10. Leia um valor inteiro N. Este valor será a quantidade de valores inteiros X que serão lidos em seguida. Mostre quantos destes valores X estão dentro do intervalo [10,20] e quantos estão fora do intervalo, mostrando essas informações conforme exemplo (use a palavra "in" para dentro do intervalo, e "out" para fora do intervalo).

Perguntas & Respostas

Wagner Cesar Vieira
wagner.vieira01@cps.sp.gov.br